

AI-Assisted Orthopedic Fracture Detection & Alignment Analysis

*A Machine Learning Approach to Fracture Localization and Displacement Quantification
Using YOLOv8*

Prepared by: Ruchir Padhye (25B2251)

Indian Institute of Technology Bombay

Summer of Code (SoC 26) — Healthcare AI Vision

2026

Table of Contents

1. Introduction & Problem Statement 3

2. Proposed Solution & Methodology 4

 2.1 Dataset 4

 2.2 Data Cleaning & Preprocessing 4

 2.3 Detection Model Architecture 4

 2.4 Alignment Analysis Methodology 4

3. Results 5

 3.1 Detection Performance (v1 Baseline) 5

 3.2 Failure Mode Analysis 6

 3.3 Extended Training & Augmentation (v2) 7

 3.4 Alignment Analysis Demonstration 7

4. Code Overview 10

5. Limitations & Future Work 10

6. Conclusion 10

References 11

1. Introduction & Problem Statement

Reading X-rays for fractures is something radiologists do every day, but it's not a solved problem. A missed or delayed fracture diagnosis still happens often, especially in busy emergency places or wherever a specialist radiologist isn't immediately available. That's the gap I wanted to explore: could a computer-vision model act as a second pair of eyes, flagging suspicious regions for a doctor to double-check, without trying to replace their judgment?

There's also a second layer to this that most fracture-detection projects skip over. Whether a fracture just needs a cast or needs surgery usually comes down to how displaced or angulated the broken fragments are and not just whether a fracture exists. Most detection-only systems stop at drawing a box around the fracture, which tells you it's there but not how badly out of alignment it is, and that's often the more clinically important number.

So, I set out to tackle both halves:

- Fracture Detection — train a YOLOv8 object detection model to localize and classify fractures across six anatomical regions: elbow, fingers, forearm, humerus, shoulder, and wrist.
- Alignment Analysis — design and validate a way to quantify how displaced and angulated a fracture is, using a key point-based (pose estimation) approach, since a plain bounding box can't tell you that.

Everything here was built in Google Colab using the Ultralytics YOLOv8 framework, with a dataset pulled from Kaggle (originally a Roboflow export). The rest of this report walks through the methodology, the results, including an honest look at where the model struggles and why, and the alignment analysis approach, along with a working demonstration of it.

2. Proposed Solution & Methodology

2.1 Dataset

I used the “Bone Fracture Detection: Computer Vision Project” dataset from Kaggle (uploaded by pkdarabi), which is originally a Roboflow export (project: veda/bone-fracture-detection-daoon, version 4). It has 3,631 training images plus validation and test splits, already in YOLOv8 bounding-box format, across six categories: elbow, fingers, forearm, humerus, shoulder, and wrist. About half the training images turned out to be background/negative examples (X-rays with no fracture at all). I checked a sample of these visually to make sure they were genuine negatives spread across all body regions and not just a chunk of unlabeled data, and they looked okay.

2.2 Data Cleaning & Preprocessing

Before training anything, I found two issues in the raw data that needed fixing:

- Class inconsistency — the data.yaml listed 7 classes instead of the 6 I expected. When I counted instances per class, ‘humerus fracture’ had only 3 labels while a class just called ‘humerus’ had 311. These were obviously the same category, just labeled inconsistently across different annotation batches (someone probably dropped the word “fracture” on part of the dataset). I merged them into one ‘humerus fracture’ class (314 instances) and re-indexed the label files across train, valid, and test, updating data.yaml to match.
- Label sanity check — before trusting the annotations, I plotted a sample of ground-truth boxes over their source images. This is where I noticed something that mattered later: boxes for the ‘positive’-suffixed classes (elbow, fingers, wrist) were drawn much more loosely, covering large chunks of the limb, while boxes for the ‘fracture’-suffixed classes (forearm, humerus, shoulder) were tight and precise. I didn’t think much of it at the time, but it turned out to explain a lot of the model’s performance gap (Section 3.2).

2.3 Detection Model Architecture

I went with YOLOv8-small (yolov8s) as the detection backbone. It’s a reasonable middle ground: enough capacity to learn from the data without overfitting badly on a dataset this size (~3,600 images), and light enough to train more than once in a single Colab session on a T4 GPU, which mattered, since I ended up needing two runs.

For the baseline run (v1): 100 epochs, image size 640×640, batch size 16, early stopping patience of 20. In practice it converged early — training stopped at epoch 33 once validation performance hadn’t improved for 20 straight epochs.

2.4 Alignment Analysis Methodology

A bounding box and class label can tell you a fracture exists and roughly where but not how badly displaced or angulated it is, which is what actually matters for treatment. To get at that, I designed a key point-based approach using YOLOv8-pose, structured like this:

- Four key points per fracture — two marking the axis of the bone fragment above the break, two marking the axis of the fragment below it.
- From those four points, the two fragment axis vectors come out of simple vector subtraction — no model needed for this part, it's just geometry.
- Angulation is the angle between the two axis vectors, reported as deviation from a perfectly straight 180°, so 0° angulation means the fragments are perfectly lined up.
- Displacement is the perpendicular distance from the fracture site: the midpoint between the two near-break key points, to where the proximal fragment's axis line would extend to.

I scoped this specifically to forearm and humerus fractures, since both are long bones with one clear, unambiguous shaft, which ultimately makes placing four key points geometrically well-defined. Fingers are too small and have too many joints for this scheme to make sense, and the shoulder doesn't really have a single clean shaft axis either, so both were left out of the alignment task.

3. Results

3.1 Detection Performance (v1 Baseline)

Here's what the baseline model came out with on the validation set:

Class	Precision	Recall	mAP50	mAP50-95
All classes	0.321	0.309	0.257	0.093
Elbow positive	0.177	0.241	0.080	0.019
Fingers positive	0.198	0.167	0.120	0.043
Forearm fracture	0.488	0.488	0.449	0.181
Humerus fracture	0.685	0.483	0.564	0.189
Shoulder fracture	0.286	0.400	0.276	0.108
Wrist positive	0.092	0.071	0.055	0.016

There's an obvious split here — the 'fracture'-suffixed classes (forearm, humerus, shoulder) do noticeably better than the 'positive'-suffixed ones (elbow, fingers, wrist). I dug into why in the next section.

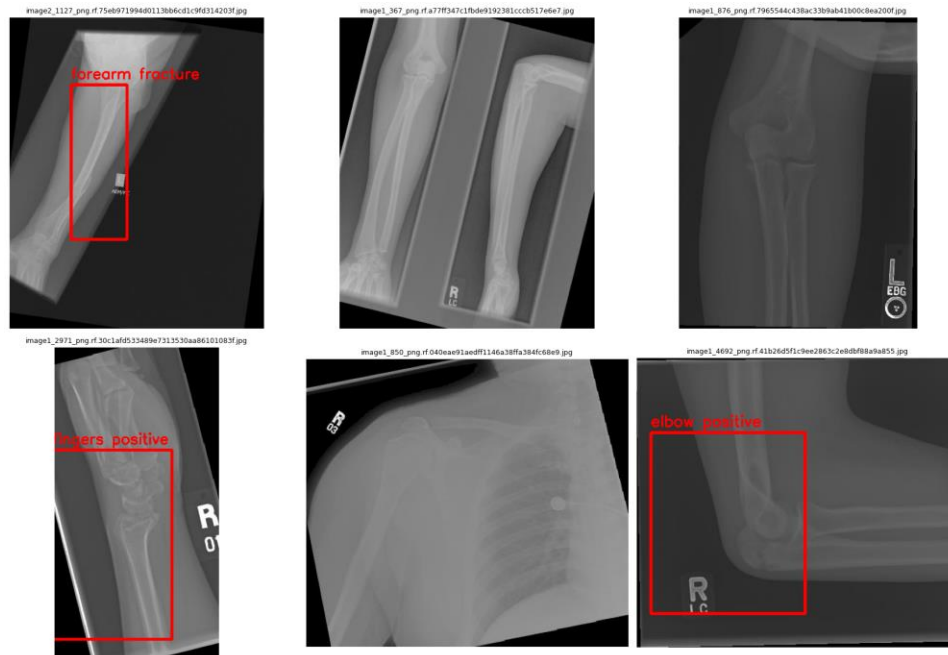


Figure 1: Ground-truth boxes plotted over source images, used to check the annotations before training.

3.2 Failure Mode Analysis

To figure out what was driving that gap, I looked at the confusion matrix from validation:

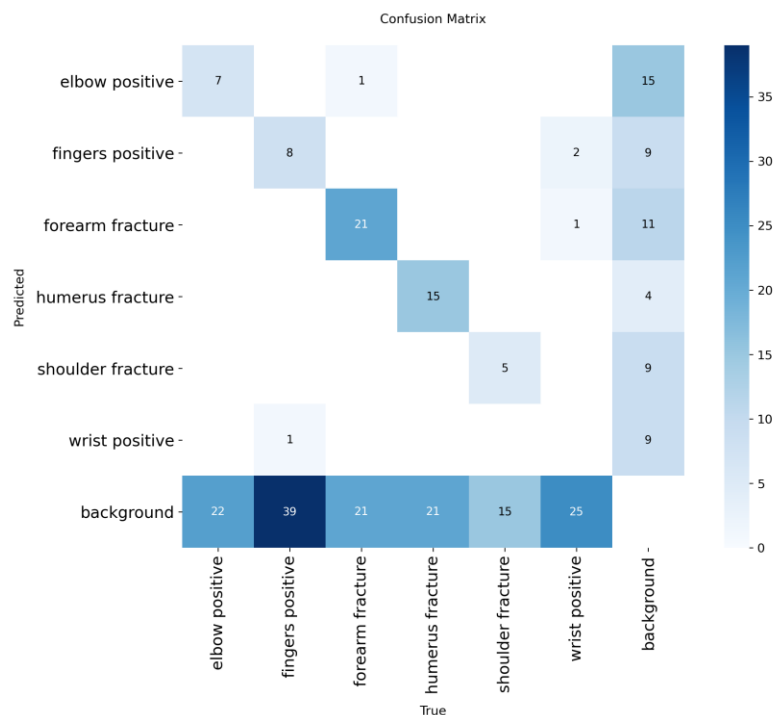


Figure 2: Confusion matrix for the v1 baseline (rows = predicted, columns = true class).

What jumps out is that almost every mistake is a missed detection: the model predicting 'background' not one class being confused for another. Fingers positive was missed 81% of the time, wrist positive

89%, elbow positive 76%, while actual cross-class mix-ups only show up as a handful of single-digit counts. This lines up with what I noticed back in Section 2.2: the ‘positive’ classes have looser ground-truth boxes, so even when the model is roughly looking at the right spot, it doesn’t hit the 50% IoU overlap that mAP50 requires to count it as correct.

I wanted to know whether this was a confidence-calibration problem (model’s close but not confident enough) or the model genuinely not picking up on the pattern at all, so I re-ran predictions at a much lower confidence threshold — 0.05 instead of the usual 0.25:

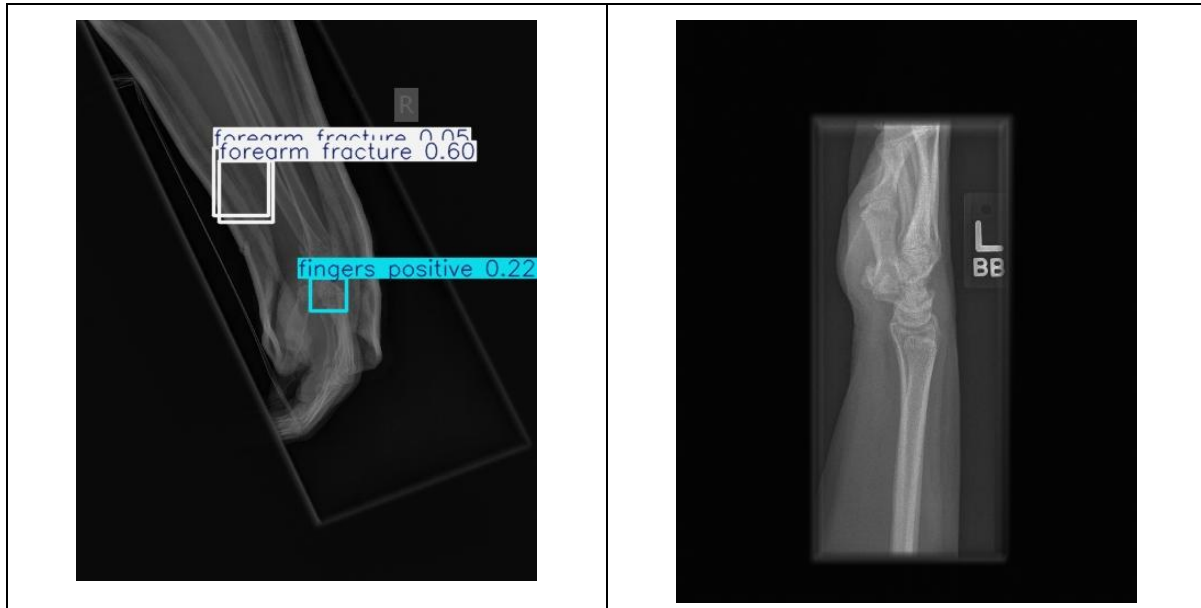


Figure 3: Predictions at $\text{conf}=0.05$ on two validation images. Left: the model correctly fires near the real fracture (forearm fracture, 0.60) but also throws out a spurious, mislabeled box (‘fingers positive’, 0.22) near the wrist. Right: a fracture that’s visually pretty clear gets no response at all, even at this low threshold.

So it’s actually both problems at once: the model does pick up real signal near some fractures, just occasionally with the wrong label, while other clearly visible fractures get nothing. Given some classes only have 300–500 labeled examples, my read is that this is a data ceiling rather than something wrong with the training setup or architecture.

3.3 Extended Training & Augmentation (v2)

To check whether that v1 ceiling was really about training time or pattern variety, I ran a second version with full 150 epochs with early stopping turned off, plus augmentation aimed at the specific issues I’d seen: rotation ($\pm 10^\circ$, since some X-rays are captured at an angle), horizontal flip (fine anatomically for limb X-rays), brightness/exposure variation (X-ray exposure varies a lot by machine), and translation.

Run	mAP50	mAP50-95
v1 (baseline)	0.257	0.0925
v2 (extended + augmented)	0.250	0.0931

v2 basically didn't move the needle: mAP50 was statistically the same (slightly lower, if anything), and mAP50-95 shifted by less than 0.001. Honestly, I think that's a useful result on its own: it tells me the ceiling here is really about dataset size and annotation consistency, not training time or lack of augmentation. Given that, I kept v1 as the model I'm reporting on, since it got basically the same result faster.

3.4 Alignment Analysis Demonstration

I didn't have time this round to get a key point-annotated dataset together, so I validated the alignment methodology from Section 2.4 in two steps instead: first with a synthetic numeric example to confirm the angulation/displacement math is geometrically correct, then visually, by placing keypoints by hand on a real validation X-ray to show what the full pipeline would actually output.



Figure 4: Alignment demo on a real forearm X-ray. Blue = proximal fragment axis, orange = distal fragment axis, magenta star = fracture site, yellow box = fracture region. Result: 177.8° between the axes (2.2° off perfectly straight) and 8.6px of lateral displacement at the break.

This confirms the geometry side of things works end-to-end — four points in, interpretable angle and displacement numbers out, overlaid right on the X-ray. What's left is training the actual YOLOv8-pose model on real annotated data, which needs a manual key point-annotation pass I haven't done yet (more on that in Section 5).

4. Code Overview

The notebook is organized into three main parts:

- Data preparation — downloading and extracting the dataset, the class-merge fix (7 classes down to the correct 6), and the label check visualizations (ground-truth boxes plus checking the background/negative images).
- Detection model — the YOLOv8s training runs (v1 and v2), validation metrics, the confusion matrix, and the low-confidence prediction check I used to figure out the failure modes.
- Alignment analysis — the geometry function itself, a synthetic worked example to check the math, and the visualization that draws fragment axes, fracture site, and bounding region straight onto a real X-ray.

Everything was trained on a Tesla T4 GPU in Google Colab using the Ultralytics YOLOv8 package. I made sure to download the checkpoints (best.pt) after each run, since Colab disconnects if you're not careful, and losing a finished training run to that would've been a rough way to lose a few hours.

5. Limitations & Future Work

- Dataset size — 300–500 labeled instances per class isn't a lot for a 6-class detector. The v1/v2 comparison points to this, along with annotation consistency, as the main thing capping performance right now.
- Annotation quality — the 'positive'-suffixed classes have noticeably looser boxes than the 'fracture'-suffixed ones. If I had more time, re-annotating those classes more tightly would probably help more than any further model tuning would.
- Alignment model not yet trained — I still need to manually annotate a key point subset (150–300 forearm/humerus images) in Roboflow. That's the immediate next step to go from a validated method to an actual trained pipeline.
- Calibration — displacement is currently just reported in pixels. To make that clinically meaningful, it needs calibrating to a real physical scale, e.g. DICOM pixel-spacing metadata or a reference marker in the image. I haven't built that part yet.
- Generalization — both models are only tested on this dataset. How they'd hold up on X-rays from a different machine, hospital, or patient population is genuinely unknown right now.

6. Conclusion

Overall, this project builds out a two-part pipeline for fracture analysis: a YOLOv8 detection model with a baseline result ($mAP_{50} = 0.257$) that I've dug into and understand well, and a fully worked out, geometrically validated alignment method using key points. I didn't just take the detection number at face value. The confusion matrix and low-confidence checks pointed clearly to a data-limited ceiling rather than a training problem, and the v2 run confirmed that by not improving anything. The alignment

side is validated analytically and visually on a real X-ray, so the path to a fully trained pipeline is clear — it just needs the keypoint annotation done.

References

- [1] Dataset: pkdarabi, “Bone Fracture Detection: Computer Vision Project,” Kaggle.
<https://www.kaggle.com/datasets/pkdarabi/bone-fracture-detection-computer-vision-project>
- [2] Original dataset source: veda, “bone-fracture-detection-daoon,” Roboflow Universe.
<https://universe.roboflow.com/veda/bone-fracture-detection-daoon>
- [3] Ultralytics YOLOv8 Documentation. <https://docs.ultralytics.com/>
- [5] Roboflow Universe. <https://universe.roboflow.com/>